3 MySQL笔记

0 总体介绍

北梦测的课程特点

1. 企业实操
2. 面试题
    - 融合: 课上 课后
    - 级别:
        - 了解[1], 面试中被问到的概率10%
        - 掌握[2], 面试中被问到的概率50%
        - 重点[3], 面试中被问到的概率80%以上

学习态度

认真, 努力

学习方法

- 理解
- 不理解的内容, 提问(老师/百度/AI), (带着答案问问题)

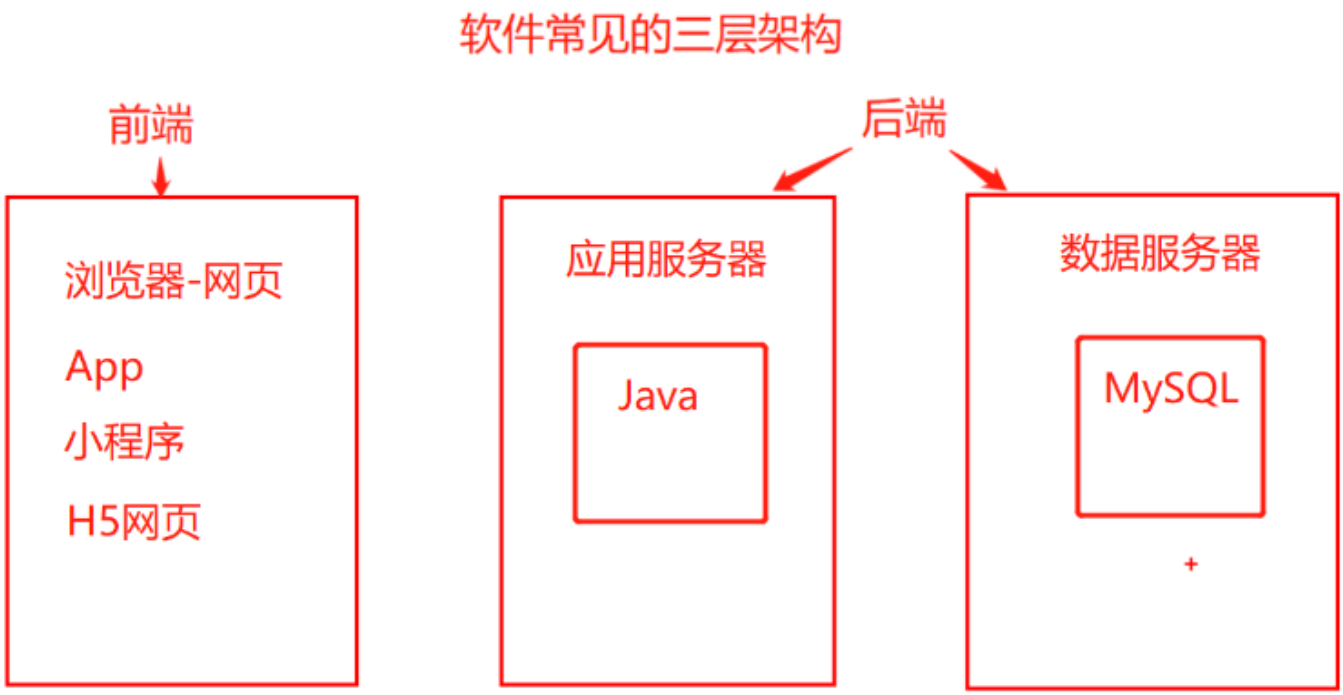
MySQL

so easy

1 数据库简介

1.1 为什么学习数据库

- 测试理论, 测试的对象, 源程序, 目标程序, 各种文档, 数据
- 几乎所有软件的数据都储存在数据库中



- 更深层的定位bug
  - 如, 刚从界面注册成功的用户, 无法登录
  - 如, 用户注册登录都没有问题

面试常见的问题: 你在测试数据的过程中发现过哪些bug

1.2 什么是数据库

数据库 (**DB**): 英文是 **database** 简单的说, 就是一个存储数据的地方

- DB 是 DBMS (Database Managment System - 数据库管理系统) 所创建的管理数据的容器

> 如, 一个企业(DBMS), 建了一个动物园(DB), 里面有各种动物 (Data)

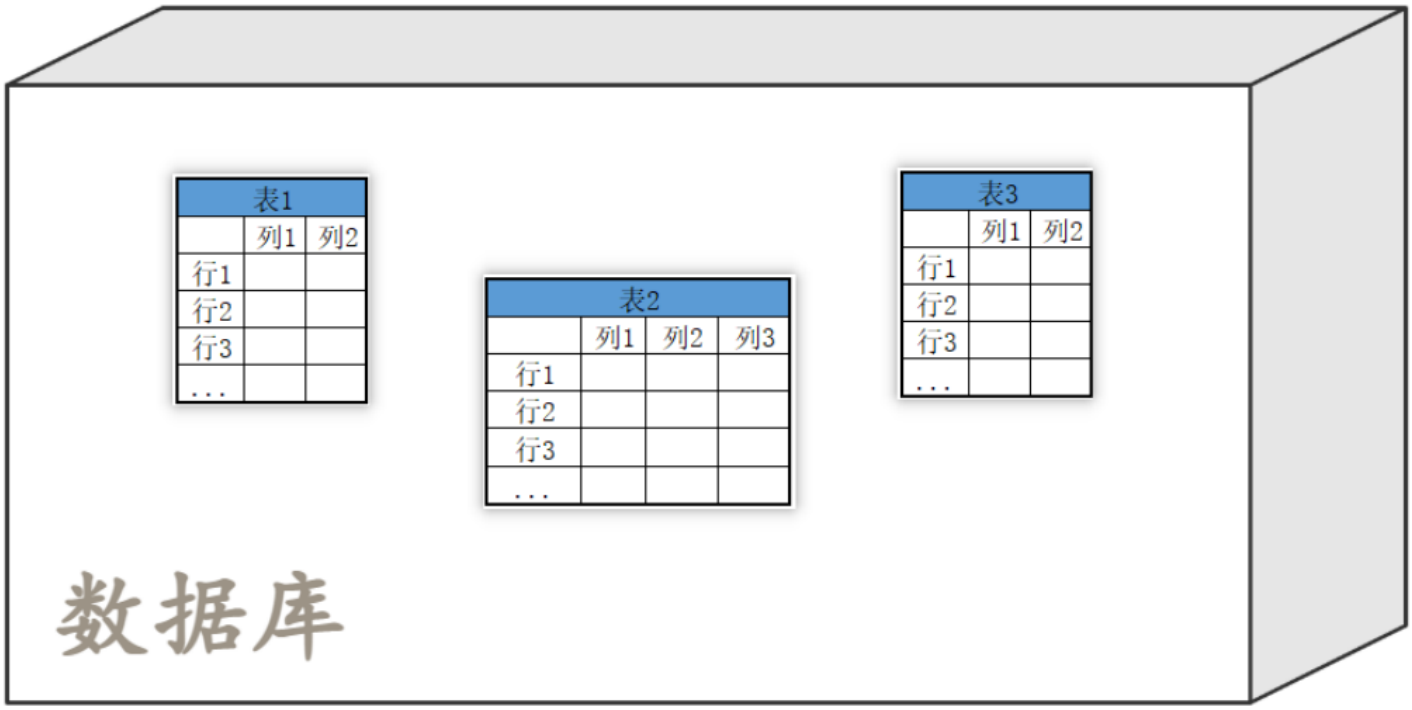
表 (**table**): 数据库中存储数据的基本单位, 数据按照分类存储到不同的表中

> 如, 为了方便分类管理各种动物, 在同一个动物园 (DB) 里, 把一群狮子放在狮子分园 (table) 中

1.3 关系型数据库

概念

简单的说, 就是由多张有关联的表组成的数据库



核心元素

- 数据库, 表的集合, 一个数据库中可以有多个表
- 表, 由行和列组成的二维表格
- 行, 记录 (一条数据)
- 列, 字段

关系型数据库的主要产品

- Oracle: 在大型项目中使用, 如: 银行, 电信等项目
- MySQL: 互联网时代使用最广泛的关系型数据库
- SQL Server: 在微软平台的项目常用
- SQLite: 轻量级数据库, 主要应用在移动平台

1.4 SQL

- SQL定义: 英文是 Structured Query Language, 结构化查询语言
- SQL是一门语言, 专门用来操作关系型数据库, 可以操作 oracle, mysql, sql server, sqlite 等等\
- SQL语言不区分大小写

1.5 MySQL安装

下载

地址: https://downloads.mysql.com/archives/community/

安装

附带安装文档+安装视频

2 Navicat

2.1 黑窗口连接数据库

- 登录
  - 本地登录
    i. 打开命令行界面 win+r, 在输入中输入cmd, 并回车
    ii. 在黑窗口中输入: mysql -uroot -p 并回车
    iii. 输入root用户的登录密码
  - 远程连接

    > - MySQL出于安全考虑, 默认关闭了远程连接权限, 需要提前打开

> 常用命令
>
> - 显示所有的数据库: show databases;
> - 使用指定的数据库: use 数据库名字;
> - 查看指定数据库的所有表: show tables;

2.2 Navicat连接Mysql

🔖 **背景:**

- **学习视频** `07 mysql-2-Navicat-2-Navicat连接数据库` **中会提到如何使用Navicat连接数据库, 但没有软件的解压和安装过程说明**
- **学习视频** `07 mysql-2-Navicat-2-Navicat连接数据库` **中连接的是本机(127.0.0.1)安装的MySQL, 为了省去同学们安装MySQL的过程, 以后统一使用虚拟机中的MySQL即可**

目标: 拥有属于自己的MySQL学习环境

内容: 推荐使用虚拟机1

- 虚拟机1的应用3: MySQL的访问和练习

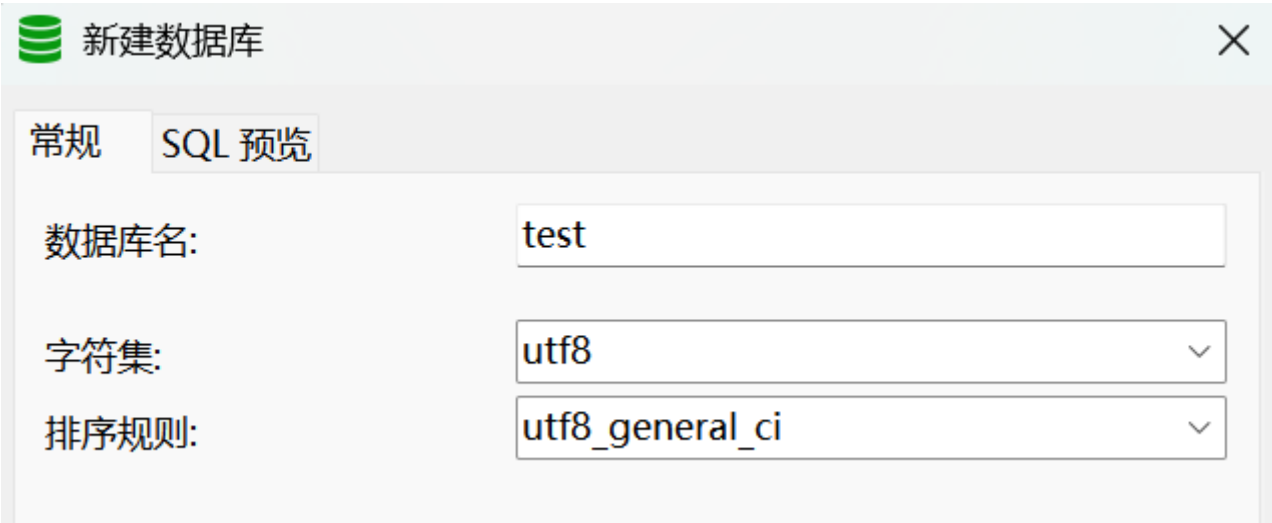
注意

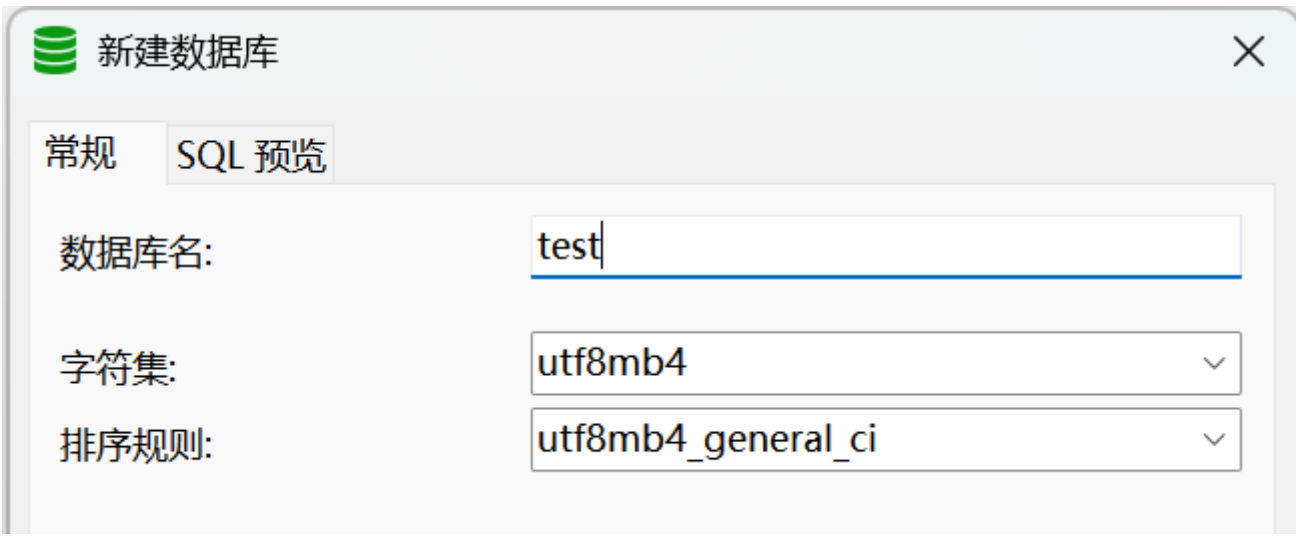
- api_test这个数据库是后面学习api1.0这个软件的时候会用到的数据库
- mydb这个数据库是后面学习api_server这个软件的时候会用到的数据库
- 系统自带的数据库不允许操作, 如: information_schema, mysql, performance_schema, sys

2.3 Navicat创建数据库

步骤

1. 双击连接名，打开连接，在连接名上，右键 -> 新建数据库
2. 填入相关信息



> 注意:
>
> - 数据库名不要用中文
> - 字符集和排序规则优先选择 `utf8和utf8_general_ci` , 如果没有这个可选的设置项, 可以选择 `utf8mb4和utf8mb4_general_ci` , 如下图所示
>
> 
>

3 SQL 基础

3.1 MySQL常用数据类型

- **整数** (**int**)
  - 有符号范围 (-2147483648, 2147483647)
  - 无符号范围 (0，4294967295), 如: int unsigned, 代表设置一个无符号的整数
- **小整数** (**tinyint**)
  - 有符号范围 (-128,127)
  - 无符号范围 (0,255), 如: tinyint unsigned, 代表设置一个无符号的小整数
- **小数** (**decimal**)
  - 如 decimal(5,2) , 表示共存5位数，小数位占2位 (不能超过2位)，整数位占3位
- **字符串** (**varchar**)
  - 如 varchar(3) , 表示最多存 3 个字符 , 一个中文或者字母都占一个字符
- **日期时间** (**datetime**)
  - 范围 (1000-01-01 00:00:00 ~ 9999-12-31 23:59:59) , 如: '2020-01-01 12:29:59'
- **文本** (**text**)
  - 用于存储一段无法确定长度的文本内容

面试题

- 数据库中有哪些常见的数据类型[1]
  - 整数, 小数, 字符串, 日期, 文本
- 有没有发现过和数据类型有关的bug[1]
  - 有
  - 我记得当时开发的是一个论坛项目, 在一次开发评审会议上
  - 后端设计的数据表里, 文章标题用的数据类型是 varchar(10)
  - 我做了调研, 发现10个字符根本不够, 后来建议改为 20个字符的限制

3.2 创建表

语法

代码块

```
# 语法格式
create table 表名 (
```

```
        字段名 数据类型,
        字段名 数据类型
    );
```

创建单个字段的表

代码块

```
# 例 1: 创建表 a, 字段要求:name(姓名), 数据类型: varchar(字符串), 长度为 10
 create table a (name varchar(10));
```

创建多个字段的表

代码块

```
# 例 2: 创建表 b, 字段要求有: id, name, age
create table b (
  id int,
  name varchar(20),
  age tinyint unsigned
);
```

> Navicat中的注释
>
> - 单行
> - #
> - --加上英文空格, 再跟上注释的内容
> - 多行
> - /* 注释内容 */

3.3 insert 插入数据

语法: insert into 表名 values (值1, 值2, 值3);

代码块

```
# 例1: 往b表里, 插入一条记录
insert into b values (0, "张飞", 30);

```

```
# 例2: 往b表里, 插入多条记录
INSERT INTO b
VALUES
  (1, "关羽", 31),
  (2, "刘备", 32);
```

指定字段插入的语法: insert into 表名 (字段名1, 字段名2) values (值1, 值2);

代码块

```
insert into b (id, name) values (3, "吕布");
```

面试题

插入数据的SQL命令是什么[1]

- insert into 加表名, 加values, 括号, 括号里写各个字段名

什么时候需要插入数据[1]

- 没有硬件设备: 如, 查询商品出库信息, 如果没有出库信息, 就可以插入一条出库的数据
- 需要付款的时候: 如, 游戏客户端上有个按钮, 可以查询游戏皮肤, 就可以插入一条相关数据, 把账号和拥有的皮肤id等信息关联起来

3.4 select查询数据

生活中的软件需要查询数据的场景

- 购物类型的软件: 搜商品, 看详情, 看评价, 看物流信息, 看各种订单
- 学生管理系统: 搜学生, 看班级信息, 看成绩
- 聊天系统: 搜附近, 搜在线, 看朋友圈, 看对方详情, 看聊天记录

查询所有字段

语法: select * from 表名;

代码块

```
# 查询b表所有字段
select * from b;
```

查询指定字段

语法: select 字段名1, 字段名2 from 表名;

代码块

```
# 查询b表的name,age字段
select name, age from b;
```

3.5 update修改数据

语法: update 表名 set 字段=值, 字段=值 where 条件

- 没有where条件时

代码块

```
# 例 1: 修改表 b, 所有人的年龄(age 字段)改为 50
update b set age = 50;
```

- 带有where条件的update语句

代码块

```
# 例 2: 修改表 b,
# id 为 3 的记录,
# 姓名(name 字段)改为 狄仁杰, 年龄(age 字段)改为 20
update b set name = "狄仁杰", age = 20 where id = 3;
```

3.6 delete删除数据

语法: delete from 表名 where 条件

代码块

```
# 例 1: 删除表 b 中 id 为 3 的记录
delete from b where id = 3;
# 例 2: 删除表 b 中的所有数据
delete from b;
```

3.7 truncate删除数据

语法: truncate table 表名;

删除表中所有的数据

3.8 drop删除表

语法1: drop table 表名;

删除表(包括表结构+表数据)

语法2: drop table if exists 表名;

如果表存在, 则删除表, 如果表不存在, 则什么都不做, 也不会报错

4 字段约束

语法格式

代码块

```
create table b (
  字段名1 数据类型 约束,
  字段名2 数据类型 约束,
  字段名3 数据类型
);
```

常用约束

- 主键 primary key, 表示值是唯一的, 不可以重复, auto_increment代表自动增长
- 非空 not null, 此字段不允许填写空值
- 默认值 default, 当不填写此字段的值时, 会使用这个默认值

示例

代码块

```
create table b (
  id int primary key auto_increment,
  name varchar(20) not null,
```

```
    age tinyint unsigned default 18
  );
  # 插入一条数据，不写id，age使用默认值
  insert into b(name) values("杜甫");
  # 插入一条数据，不写id，age使用自定义的值
  insert into b(name, age) values("白居易", 20);
```

面试题

delete / truncate / drop 三者的区别[3]

- delete 删表数据，保留主键记录和表结构
- truncate 删表数据，会同时删除主键记录，保留表结构
- drop 删表，包括表结构，主键记录，表数据全部删除

5 条件查询

5.1 数据准备

代码块

```
# 创建学生表
create table student(
    id INT PRIMARY KEY auto_increment,
    name VARCHAR(20) not null,
    sex CHAR(1),
    age TINYINT UNSIGNED,
    class TINYINT UNSIGNED,
    position VARCHAR(5)
);

#插入数据
INSERT INTO student
VALUES
(1, "白起", "男", 40, 1, "坦克"),
(2, "猪八戒", "男", 80, 2, "坦克"),
(3, "花木兰", "女", 22, 3, "战士"),
(4, "狂铁", "男", 30, 1, "战士"),
(5, "娜可露露", "女", 20, 2, "刺客"),
(6, "李白", "男", 28, 3, "刺客"),
(7, "妲己", "女", 18, 1, "法师"),
(8, "周瑜", "男", 25, 2, "法师"),
(9, "孙尚香", "女", 18, 3, "射手"),
```

```
(10, "鲁班七号", "男", 16, 3, "射手"),
(11, "庄周", "男", 21, 3, "辅助"),
(12, "瑶", "女", 18, 3, "辅助");
```

5.2 where

语法

select * from 表名 where 条件;

代码块

```
# 例1: 查询 student 表中 id 为 1 的数据
select * from student where id = 1;

# 例2: 查询 student 表中, age 为 30的姓名 name, 班级 class
select name,class from student where age = 30;
```

> 补充:
>
> where 关键字也可以在 update 和 delete 语句中使用

工作中用到 where 的场景

- 搜商品, where 在商品名或者商品介绍中包含搜索的关键字
- 看详情, where id 为 具体某一个商品
- 搜学生, where id或者是姓名 为具体某一个学生
- 搜附近, where 距离 小于 多少公里

5.3 比较运算符

代码块

```
# = 表示等于
select * from student where age = 25;
# < 表示小于
select * from student where age < 25;
# <= 表示小于等于
select * from student where age <=25;
# > 表示大于
select * from student where age > 25;
```

```
# >= 表示大于等于
select * from student where age >= 25;
# != 表示不等于
select * from student where age !=25;
```

5.4 逻辑运算符

- and 与
- or 或
- not 非

代码块

```
# 例 1: 查询 age 年龄小于 30, 并且 sex 性别为 女 的同学记录
select * from student where age < 30 and sex = "女";

# 例 2: 查询 sex 性别为 女 或者 class 班级为 1班的学生记录
select * from student where sex = "女" or class = 1;

# 例 3: 查询 position 职业非 辅助 的学生记录
select * from student where not position = "辅助";
```

5.5 模糊查询

生活中的场景

- 精确查询 (等于)
  - 开车出入停车场, 车牌号经过精确查询后进行缴费
  - 去药店买药, 报精确会员手机号, 以便会员积分
- 模糊查询 (包含)
  - 百度, 搜索关键字
  - 淘宝, 根据关键字搜索商品

使用

like 实现模糊查询

- % 代表任意多个字符

- _ 代表任意一个字符

代码块

```
# 查询 name 以"狂"开头的学生数据
select * from student where name like "狂%";
select * from student where name like "狂_";
# 查询 name 以"铁"结尾的学生数据
select * from student where name like "_铁";
select * from student where name like "%铁";
# 查询 name 中含有"周"的学生数据
select * from student where name like "%周%";
```

在工作中用的最多的情况

就是 like "%被含有的内容%"

5.6 范围查找

- 通过 between ... and ..., 在连续的范围内进行查找

代码块

```
# 例1: 查询 age 年龄为 25 至 30 的学生记录
select * from student where age >= 25 and age <= 30;
select * from student where age between 25 and 30;
```

- 通过 in, 在非连续的范围内进行查找

代码块

```
# 例2: 查询 position 职业 是 "刺客"或"射手"或"辅助"的学生
select * from student where position = "刺客" or position = "射手" or position = "辅助";
select * from student where position in ("刺客","射手","辅助");
```

5.7 null

代码块

```
# 数据准备
```

```
insert into student values (13, "干将", null, null, 4, "法师");

# 例1: 查询 sex 性别 为 空 的学生记录
select * from student where sex is null;

# 例2: 查询 sex 性别 非 空 的学生记录
select * from student where sex is not null;
```

> **注意**
>
> - null 在 SQL 中代表空, 不是0, 也不是"", 而是什么也没有
> - is null 判断为空
> - is not null 判断非空
> - null 不能用比较运算符判断

6 别名

表的别名

代码块

```
# 通过 as 给表 student 起一个别名
select * from student as stu;
# as 可以省略
select * from student stu;
```

字段的别名

代码块

```
 # 通过 as 给字段起一个别名, as 可以省略
 SELECT
   id as ID,
   name as 姓名,
   sex as 性别,
   age as 年龄,
   class 班级,
   position 职业
 FROM
   student;
```

7 排序

语法

- 升序(从小到大): order by 字段名 asc，其中asc是默认排序, 可以不写
- 降序(从大到小): order by 字段名 desc

单字段排序

代码块

```
# 查询student学生数据，按age年龄从小到大排序
select * from student order by age asc;
select * from student order by age;
# 查询student学生数据，按age年龄从大到小排序
select * from student order by age desc;
```

多字段排序

代码块

```
# 查询student学生数据，按age年龄从小到大排序
# 当年龄相同时，再按id从小到大排序
select * from student order by age, id;
```

当一个select语句中出现了where和order by

语法: select * from 表名 where 条件 order by 字段1, 字段2;

代码块

```
# 查询student所有的男生数据，按班级升序，班级相同时，再按年龄升序
SELECT
        *
FROM
        student
WHERE
        sex = "男"
ORDER BY
        class,
```

```
10          age;
```

8 聚合函数

8.1 count 计数

count(字段名) 或者 count(*)

代码块

```
1   # 查询student表的学生总数
2   select count(*) from student;
3   select count(id) from student;
4   # count(*)和count(id)区别
5   # 查询结果是一样的，效率不同，大量数据时，后者更快
6
7   # 查询student表的女生总数
8   select count(*) from student where sex = "女";
9   select count(id) from student where sex = "女";
```

补充: distinct 去重

代码块

```
1   # 查询 student表的班级总数
2   select class from student;  -- 只查出 class 这一列的数据
3   select count(class) from student;  -- 查出的是有班级数据的数据总条数
4   select distinct class from student; -- 去重后都有什么班级
5   select count(distinct class) from student;  -- 去重后，统计班级总数
```

8.2 max 最大值

语法: max(字段名), 查询指定字段里的最大值

代码块

```
1   # 查询 student 中的最大年龄
2   select max(age) from student;
3   # 查询 student 中女生的最大年龄
4   select max(age) from student where sex = "女";
```

8.3 min 最小值

语法: min(字段名), 查询指定字段里的最小值

代码块

```
# 查询 student 中的最小年龄
select min(age) from student;
# 查询 student 中女生的最小年龄
select min(age) from student where sex = "女";
```

8.4 sum 求和

语法: sum(字段名), 指定字段的值求和

代码块

```
# 查询 student 表中 年龄的总和
select sum(age) from student;
# 查询 student 表中 女生年龄的总和
select sum(age) from student where sex = "女";
```

8.5 avg 求平均数

语法: avg(字段名), 求指定字段的平均值

代码块

```
# 查询 student 表中 年龄的平均值
select avg(age) from student;
# 查询 student 表中 女生年龄的平均值
select avg(age) from student where sex = "女";
```

> 如果值中含有 null, null是不参与计算平均值的

聚合函数不能用到 where 的条件中[1]

9 数据分组

9.1 分组的实现

生活中需要分组统计的场景

- 老师查全班成绩, 需要统计及格和不及格分别有多少人
- 记录日常开销的软件, 我们可能需要统计本月收入和开支总计多少钱

实现

- 分组 一般需要配合 聚合函数 来使用
- group by 字段名
- select 聚合函数 from 表名 group by 字段名;
- select 字段名, 聚合函数 from 表名 group by 字段名;
- select 字段名, 聚合函数 from 表名 where 条件 group by 字段名;

代码块

```
# 按不同性别分组，来查询 student 表中男女的总数
select count(*) from student where sex = "男";
select count(*) from student where sex = "女";

select count(*) from student group by sex;
select sex, count(*) from student group by sex;

# 按不同性别分组，来查询 student 表中3班的男女的总数
select sex,count(*) from student where class = 3 group by sex;
```

- 多字段分组

代码块

```
# 按不同班级,不同性别分组，来查询 student 表中男女的总数
select class, sex, count(*) from student group by class, sex;
# 按不同班级，不同职业分组，来统计 student 表中的总数
select class, position, count(*) from student group by class, position;
```

- 分组后的排序
- select 字段名, 聚合函数 from 表名 where 条件 group by 字段 order by 字段;

代码块

```
# 按不同班级不同性别进行分组，按班级降序，来统计 student 表中男女的总数
```

```
 select class, sex, count(*) from student group by class, sex order by class desc;

 # 按不同班级不同性别进行分组，按班级降序，来统计 student 表中成年的男女的总数
 select class, sex, count(*) from student where age >= 18 group by class, sex order by class desc;
```

9.2 分组后的数据筛选

- group by 字段 having 条件

代码块

```
# 查询 student 表的男生的总数
# where，先筛选，再统计
select count(*) from student where sex = "男";
# having，先分组，再筛选，再统计
select count(*) from student group by sex having sex = "男";
```

- having 配合聚合函数的使用

代码块

```
# 查询 student 表，统计班级人数大于3人的班级是？
select class from student group by class having count(*)>3;
```

面试题

where与having筛选的区别[2]

- 筛选逻辑
  - where 对原始数据进行筛选，效率较低
  - having 是对group by 这个分组后的数据进行筛选，效率较高
- having可以使用聚合函数，where不可以

10 分页

- limit 开始行，总数
  - 如，limit 0，3 表示从第1行开始，显示3行

- 如果是从第1行开始, 那么开始行这个0可以省略, 那么 limit 0, 3 可以写成 limit 3

代码块

```
# 查询 student 表中前三条记录
select * from student limit 3;
select * from student limit 0, 3;
# 查询从第4条开始的3条数据(其实就是按照每页3条, 这是第二页)
select * from student limit 3, 3;
# 按照每页4条, 展示第2页
select * from student limit 4, 4;
```

- limit 总是出现在select语句的最后

面试题

SQL语句的综合书写顺序[2]

1. select
2. from
3. where
4. group by ... having ...
5. order by
6. limit

11 连表查询

11.1 总体介绍

student

id	name	sex	age	class	position
1	白起	男	40	1	坦克
2	猪八戒	男	80	2	坦克
3	花木兰	女	22	3	战士
4	狂铁	男	30	1	战士
5	娜可露露	女	20	2	刺客
6	李白	男	28	3	刺客
7	妲己	女	18	1	法师
8	周瑜	男	25	2	法师
9	孙尚香	女	18	3	射手
10	鲁班七号	男	16	3	射手
11	庄周	男	21	3	辅助
12	瑶	女	18	3	辅助
13	干将			4	法师

course

id	name
1	大学语文
2	火系魔法导论
3	高等恋爱理论

sc

s_id	c_id	score
8	2	98
5	2	66
2	1	20
2	3	88
6	1	99

> 如何查询猪八戒的分数?

代码块

```
# 数据准备

# 课程数据
create table course(
    id INT PRIMARY KEY auto_increment,
    name VARCHAR(20) not null
);

INSERT INTO `course` VALUES (1, '大学语文');
INSERT INTO `course` VALUES (2, '火系魔法导论');
INSERT INTO `course` VALUES (3, '高等恋爱理论');

# 成绩数据
create table sc(
    s_id INT,
    c_id INT,
    score INT
);

INSERT INTO `sc` VALUES (8, 2, 98);
INSERT INTO `sc` VALUES (5, 2, 66);
INSERT INTO `sc` VALUES (2, 1, 20);
INSERT INTO `sc` VALUES (2, 3, 88);
INSERT INTO `sc` VALUES (6, 1, 99);
```

11.2 内连接

- **语法**
  - select * from 表1 inner join 表2 on 表1.字段 = 表2.字段
  - 内连接最重要的是: 找对两张表要关联的字段

代码块

```
select * from a inner join b on a.id = b.id;
```

- 用 student表和sc表进行内连接查询得到的结果

代码块

```
# 没有字段关联的情况下
select * from student inner join sc;
# 在找对了关联字段的情况下 关联字段是 student表中的id 和 sc表中的s_id
select * from student inner join sc on student.id = sc.s_id;
```

- 内连接查询, 显示指定字段

代码块

```
# student表与sc内连接, 只显示 name, c_id, score
SELECT
        student.name,
        sc.c_id,
        sc.score
FROM
        student
        INNER JOIN sc ON student.id = sc.s_id;

# 别名在连表查询中的应用
SELECT
        s.name 姓名,
        sc.c_id 课程ID,
        sc.score 成绩
FROM
        student s
        INNER JOIN sc ON s.id = sc.s_id;
```

- 带有 where 的内连接
  - **语法**: select * from 表1 inner join 表2 on 表1.字段 = 表2.字段 where 条件

代码块

```
# 查询猪八戒的成绩，要求显示姓名，课程号，成绩
SELECT
        s.name 姓名,
        sc.c_id 课程号,
        sc.score 成绩
FROM
        student s
        INNER JOIN sc ON s.id = sc.s_id
WHERE
        s.name = "猪八戒";
```

- 多表内连接

代码块

```
# 查询学生选课成绩，要求显示姓名，课程名，成绩
SELECT
        s.name 姓名,
        c.name 课程名,
        sc.score 成绩
FROM
        student s
        inner join sc on s.id = sc.s_id
        inner join course c on sc.c_id = c.id;

# 查询李白选课成绩，要求显示姓名，课程名，成绩
SELECT
        s.name 姓名,
        c.name 课程名,
        sc.score 成绩
FROM
        student s
        inner join sc on s.id = sc.s_id
        inner join course c on sc.c_id = c.id
WHERE
        s.name = "李白";
```

写SQL三步法

1. 搭框架
    - 基本的select语句，如果涉及多表，把相应的多表连接也写出来

2. 看条件
   - 决定where后面的具体条件
3. 显示的字段
   - select后面要显示的字段

代码块

```
# 查询所有学生的"火系魔法导论"课程成绩，要求只显示姓名，课程名，成绩

# 1.搭框架
SELECT
  *
FROM
  student s
  inner join sc on s.id = sc.s_id
  inner join course c on c.id = sc.c_id
# 2.看条件
SELECT
  *
FROM
  student s
  inner join sc on s.id = sc.s_id
  inner join course c on c.id = sc.c_id
WHERE
  c.name = "火系魔法导论"
# 3.显示的字段
SELECT
  s.name 姓名,
  c.name 课程名,
  sc.score 成绩
FROM
  student s
  inner join sc on s.id = sc.s_id
  inner join course c on c.id = sc.c_id
WHERE
  c.name = "火系魔法导论"

# 三步法中 如果条件较复杂 涉及排序时
# 查询成绩最高的男生信息，要求显示姓名，课程名，成绩
SELECT
  s.name 姓名,
  c.name 课程名,
  sc.score
FROM
  student s
  inner join sc on s.id = sc.s_id
```

```
    inner join course c on c.id = sc.c_id
  where
    s.sex = "男"
  order by
    sc.score desc
  limit 1
```

11.3 左连接

语法: select * from 表1 left join 表2 on 表1.字段 = 表2.字段

代码块

```
# 查询所有学生的信息和成绩，包括没有成绩的学生
SELECT
  *
FROM
  student s
  inner join sc on s.id = sc.s_id
SELECT
  *
FROM
  student s
  left join sc on s.id = sc.s_id
```

11.4 右连接

语法: select * from 表1 right join 表2 on 表1.字段 = 表2.字段

代码块

```
# 查询所有课程的信息，包括没有成绩课程
SELECT
        *
FROM
        sc
        RIGHT JOIN course ON course.id = sc.c_id
```

11.5 小结

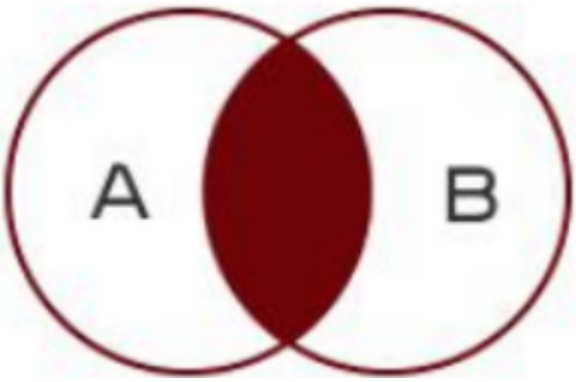
面试题

内连接, 左连接 和 右连接的区别 [3]

- 内连接, 严格显示两张表连接字段相等的数据
- 左连接, 显示左表全部数据, 包括连接字段相等的数据, 还有如果右表无数据的话会显示为null
- 右连接, 显示右表全部数据, 包括连接字段相等的数据, 还有如果左表无数据的话会显示为null

图示

- 内连接



- 左连接

- 右连接

12 自关联

代码块

```
# 数据准备

# 创建一个行政区划表，包含字段有：唯一id，区域名称，父级id
create table area (
  id int primary key,
  name varchar(10),
  pid int
);

# 插入基础数据
INSERT INTO area
VALUES
(1, "广东省", null),
(2, "浙江省", null),
(3, "广州市", 1),
(4, "佛山市", 1),
(5, "杭州市", 2),
(6, "番禺区", 3);
```

代码块

```
# 查询有什么省份，有多少个省
select * from area where pid is null;
select count(*) from area where pid is null;
```

- 自关联，是同一张表做连接查询
  - 一定要找到同一张表可关联的不同字段

代码块

```
# 查询广东省的所有城市
SELECT
  *
FROM
  area a1
  inner join area a2 on a1.id = a2.pid
WHERE
  a1.name = "广东省";
```

> 补充
>
> 自关联是一种关联的思路，并不属于关联方式

自关联的使用场景

- 同一张表中, 有父子关系的字段时就可能需要使用
  - 例如, 行政区划表
  - 例如, 电商系统中的商品分类表

13 子查询

- 子查询是嵌套到主查询里面的
- 子查询一般作为主查询的
  - 条件

代码块

```
# 例1: 查询大于平均年龄的学生记录
select avg(age) from student;
select * from student where age > 28;

select * from student where age > (select avg(age) from student);
```

  - 数据源

代码块

```
# 例2: 在上一个例子的结果中, 把职业是"坦克"的数据找出来
SELECT
        *
FROM
        (select * from student where age > (select avg(age) from student)) ss
WHERE
        position = "坦克"
```

14 存储过程

定义

存储在数据库服务端的SQL语句

作用

- 批量处理

  - **面试题**
  - 你什么时候会用到存储过程? [3]
    - 批量添加数据的时候会用到
      - 做性能测试时, 会使用存储过程帮我加海量用户数据

代码块

```
# 创建用户表, 字段: id, name, 手机号phone_number, 密码password
create table user(
    id int primary key,
    name varchar(10),
    phone_number bigint,
    password varchar(20)
);

# 创建存储过程 d, 录入10000条数据 (注意某些数据库版本不支持这种写法, 需要用22行~35行的写法)
create procedure d ()
begin  -- 存储过程的开始
    declare i int default 1; -- 定义变量i, 初始值为1
    while i <= 10000
        do  -- 接下来是需要执行的语句
            insert into user values(i, concat("张三", i), 13000000000+i, "123456");
        set i = i + 1;
    end while;
end   -- 存储过程的结束


# 创建存储过程d, 比起视频演示代码, 主要增加了分隔符修改语句, 避免了因为分号结束符导致中途退出执行的错误
DELIMITER $$  -- 把分隔符从分号改成$$

CREATE PROCEDURE d()
BEGIN
    DECLARE i INT DEFAULT 1;  -- 定义变量i, 初始值设为1

    WHILE i <= 10000 DO
        -- 执行插入操作
        INSERT INTO user VALUES (i, CONCAT("张三", i), 13000000000+i, "123456");
        SET i = i + 1;  -- 让循环变量递增
    END WHILE;
END$$  -- 存储过程结束, 使用新的分隔符$$
```

```

DELIMITER ;  -- 再把分隔符改回分号


# 调用存储过程
call d();
```

- 对功能做一些极限验证时，如测试海量商品时分页功能和分页跳转功能时，需要加数据

- 重复使用

代码块

```
# 调用存储过程
call d();
```

15 非关系型数据库介绍

非关系型数据库，也称 **NoSql**，全称为 **Not Only SQL** ，是对关系型数据库的一种 **补充**

非关系型数据库的主要产品

- **Redis**
  - 优缺点：内存数据库，读写性能碾压mysql，但耗资源，存储数据量有限
  - 应用场景：
    - 需要高频变更数据时，如，文章点赞数/阅读量
    - 频繁访问数据库的高并发场景，如，秒杀
- **MongoDB**
  - 优缺点：海量数据下，性能优于mysql，但对连表和事务等复杂业务的支持有限
  - 应用场景：
    - 无复杂业务时，替代mysql
    - 拥有内存映射技术，可当作内存数据库使用

面试题

你用过redis，有没有发现过什么bug?[3]

- 有

- 缓存雪崩: 遇到一个线上bug, redis缓存在某一时刻失效, 大量请求到达mysql, 导致数据库崩溃
- 当时我的做法是建议
  a. 构建多级缓存, 部署多个redis, 避免同时过期
  b. 建议后端做请求的队列机制

16 物理删除和逻辑删除

- 物理删除: 执行的是**delete**操作, 数据从硬盘中删除
- 逻辑删除: 执行的是**update**操作, 数据还存在硬盘中

面试题

使用数据库时, 遇到过哪些bug?[3]

- 例如, 当时做的OA项目, 其中员工表在员工离职的功能上使用的是物理删除, 我觉得不妥, 因为这样就不能标识哪个员工是"二进宫"的
- 例如, 当时做的是电商项目, 其中商品表里, 商品下架功能用的是物理删除, 我觉得不妥, 因为这样的话再次上架该商品又得重新操作上架流程, 给商家带来了困扰, 属于用户体验问题